SHELL PWN Capture The FLag
(10:00AM-10:00PM , 11/04/2026) - Writeups

Team Name : SHER
Team Members : Srujan Zanjal (marius_94)
Rishi Walokar (GhostOfSparta)
Harsh Gupta (tr0j4n)

Rank : 1st
Flags Captured : 41/45
Points : 9150
Awards (Partial FLag) : 200
Total points : 9350

OSINT :

1) Garden In a Desert City (100pts)

File : garden.jpg

Solution : The image when searched on google lens was identified as Salmiya Garden located at Kuwait .

Flag : SHELL{Salmiya_Garden}

2) Saber (100pts)

FIle : saber.png

Solution : By identifying the location in the image as the Avalon Hand Car Wash via 360° Street View, I connected the clue "grave of the wielder of the Sword of Promised Victory" to King Arthur (Excalibur). This led me to King Arthur's Tomb in Glastonbury.

The resulting Plus Code for the flag is 47WP+M2 Glastonbury, United Kingdom.

Flag : SHELL{47WP+M2}

3) Land of Ice (100pts)

FIle : landofice.jpg

Solution : The image when searched on google lens , gives the exact location i.e.Landmannalaugar valley , Iceland . We need to identify the nature reserve where this was located and the nature reserve was Fjallabak

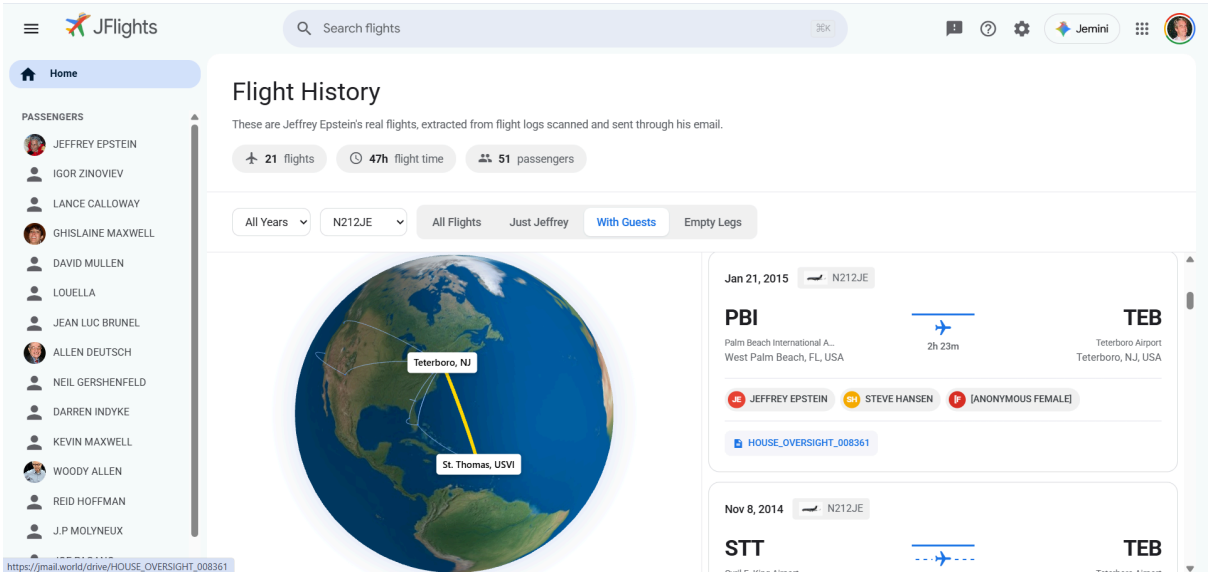
Flag : SHELL{Fjallabak}

4) Plane (600pts) (Lone Solver)

The flag format is: shell{X_A_K_C}
File : plane.jpeg

Solution : The above File contained the plane image on which it's tail tells its number i.e N212JE
When researched about this flight number the plane came out to one of the Jeffery Epstein's private plane

Then searched on Jmail (contains every data in sorted form regarding jeffery epstein) Flight section.
The description of the challenge mentioned that flight travelled in year A , from FL(Palm Beach International) to NJ(Teterboro Airport)
The primary passenger was X, accompanied by Y, an unidentified passenger, and Z, another prominent individual. At the time of the flight, Z was K years old.
After applying all this filters we got one Flight on the mentioned platform



We Got a full flight report though which we also got our required values .

A is 2015 , the year when this flight travelled
X was identified as Jefferey Epstein
Y was the unidentified passenger covered with black
Z was another passenger Steve Hansen in 1959 and his age was 56 (i.e K) in the year 2015
The total fuel consumed was C which is 7397 .

HOUSE_OVERSIGHT_008361.jpg

N212JE JEGE, LLC Passenger Manifest
Crew: LV DR NM

Date: 1 / 21 / 2015 Trip: 129

Depart: PBI Time: 2 : 43 AM / PM local

City: WEST Palm Bch State: FL

Arrive: TEB Time: 5 : 03 AM / PM local

City: TETERBORO State: N.J.

1 Passenger: JEFFREY EPSTEIN 7 Passenger:

2 Passenger: [redacted] 8 Passenger:

3 Passenger: Steve HANSEN 9 Passenger:

4 Passenger: 1 female 10 Passenger:

5 Passenger: 11 Passenger:

6 Passenger: 12 Passenger:

Gallons purchased: 1200 Fuel Burn lbs. 7397

Flight Level: FL450 Flight Time: 2 + 19

Airframe Time: Start: 8873 . 0

Today's flight: + 2 . 3

Total Airframe Time = 8875 . 3

Night: 1.0 T/O 1 Ind Approach 45 Min Flex VOR

After combining all values in the flag format the final flag is :

Flag : shell{Jeffreyepstein_2015_56_7397}

BINARY EXPLOITATION :

1) Gatekeeper (500pts)

File : gatekeeper.exe

Solution : I solved it by doing a full static reverse of gatekeeper.exe, then reconstructing the hidden decoder logic byte-for-byte.

What I looked for first

Triage:
Confirmed it is a 32-bit PE.
Extracted strings to map user-visible behavior.
From strings, identified command interface and auth clues:
Terminal banner, prompt, and commands:
help/auth/status/exit/query.
Hardcoded visible password string: s3cur3_p4ss!.
Success marker text: * ACCESS GRANTED *.
How I mapped behavior to code

Located references to important strings in disassembly:
Password prompt in gatekeeper_disasm.txt:490.
Access denied path in gatekeeper_disasm.txt:518.
Main terminal loop banner/prompt in gatekeeper_disasm.txt:581 and gatekeeper_disasm.txt:587.
Reconstructed the command dispatcher around the main loop:
help -> function at 0x40162d.
auth -> function at 0x4016ee.
status -> function at 0x401672.
query -> function at 0x401818.
exit -> terminate.
Verified the auth check compares typed password against the hardcoded s3cur3_p4ss! string, then sets an authenticated flag.
Where the hidden backdoor came from

Found a suspicious function at address 0x401460 that was not called from normal command flow.
That routine:
Reads bytes from global data near 0x405004.

Runs a loop for 0x1A bytes.
Applies transform:
out[i] = data[i] XOR (i XOR 0xA7)
Prints the decoded result using a format string at 0x406064.
Decompiled that exact transform in a quick Python snippet and decrypted the bytes.
Output was:
shell{g4t3k33p3r_br34ch3d}

Flag : shell{g4t3k33p3r_br34ch3d}

REVERSE ENGINEERING :

1) Baby XOR (100pts)

File : baby_xor.exe

Solution : The goal was to find a flag hidden behind a simple XOR operation.Since the challenge title and description emphasized a "classic bitwise operation" (XOR), we performed a prefix scan on the binary. We searched for the bytes corresponding to the string `shell{` XORed with every possible single-byte key (0-255).
We found a match in the binary's data section with the XOR key `0x5a`.
XORing the subsequent bytes with `0x5a` revealed the complete flag

Flag : shell{b4by_r3v}

2) Trust Nothing (100pts)

File : Challenge4.exe

Solution :

1. Initial Reconnaissance

Running the executable revealed a standard flag checker. To investigate further without executing the logic, a strings dump was performed on the .exe file.

The output immediately revealed several obvious flags:

shell{th1s_is_t00_e4sy}

shell{n0pe_n0t_th1s_0ne}

shell{y0u_w1sh_it_was_th1s}

However, these were paired with taunting error messages (e.g., "[-] Lol, did you really just run 'strings' and guess"), confirming the challenge hint: these were intentionally planted red herrings.

2. Spotting the True Target

Looking past the decoy strings, the actual success message ("ACCESS GRANTED! You found the true flag.") was located further down in the data section.

Directly above this success message was a suspicious string:

c2hlbGx7dHJ1NXRfbjB0aDFuZ30=

The alphanumeric characters and the trailing = sign padding immediately identified this as Base64 encoded data. This was further confirmed by the presence of the standard Base64 index table (ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789) nearby in the memory dump.

3. The Solution

Instead of reversing the comparison logic in a decompiler, the challenge was bypassed entirely by decoding the Base64 string.

Using CyberChef (or a standard terminal base64 decoder):
From Base64: c2hlbGx7dHJ1NXRfbjB0aDFuZ30=
Flag: shell{tru5t_n0th1ng}

Flag : shell{tru5t_n0th1ng}

3) Gate of Overflow

File : challenge3.exe

Solution : **Reversal Process**

1. Identifying the Target

By dumping the binary data near the string "Access Denied", we found a 18-byte sequence: `ca 5e ce c4 c5 9d 68 26 d9 6e e4 e4 1b 92 8a 82 11 77`

2. Algorithmic Analysis

We extracted the machine code at the comparison loop and identified the following algorithm:

1. The program loops through the 18 characters of the flag.
2. For each character `flag[i]`, it performs:
    - $X = (flag[i] \times 37) \pmod{256}$
    - $Y = X \oplus (0x55 + i)$
3. It then compares Y to the target byte at that index.

3. Solving the Flag

We derived the reverse operation:

- $X = Target[i] \oplus (0x55 + i)$
- $flag[i] = (X \times 37^{-1}) \pmod{256}$ (where $37^{-1} \equiv 173 \pmod{256}$)

Running the solver produced the flag `shell{wr4p_4r0und}`.

4. Verification

Running the executable with the flag confirmed its correctness:

Flag : shell{wr4p_4r0und}

4) Vault 42

FIle : vault

Solution : I solved it statically from vault, without running it under a debugger.

The workflow was:
Identify the file as a stripped 64-bit Linux ELF.
Pull strings and disassembly.
Notice two important things immediately:
A fake flag is embedded in .rodata:
SHELLCTF{gdb_w0nt_s4v3_y0u_h3r3}
The binary imports ptrace, which is the usual anti-debug trap.
Read the main validation path in the disassembly.
The key logic is simple once decompiled mentally:

The program first checks that you passed exactly 1 argument after the program name.
It requires the candidate key to be 25 bytes long.
It then compares each byte against a table stored in .data, using this transform:
$target[i]=(((input[i]\oplus 0x42)+i)\oplus 0x37)\bmod 256$

So to recover the real input, I inverted it:
$input[i]=(((target[i]\oplus 0x37)-i)\bmod 256)\oplus 0x42$

The target bytes were sitting in .data at runtime offsets corresponding to the compare table. Once I dumped those bytes

and applied the inverse transform across all 25 positions, the accepted flag came out as:
SHELLCTF{m4th_4nd_ptr4ce}
Why the debugger trick mattered: if you run it under gdb or another tracer, the ptrace check trips and it prints the fake flag instead of proceeding normally. Static analysis avoids that entirely.

Flag : SHELLCTF{m4th_4nd_ptr4ce}

5) drop

FIle : init_routine.S and flash_dump.bin

Solution : I solved it by treating init_routine.S and flash_dump.bin as a small stateful byte transform, then reversing that transform offline. The routine is ARM Cortex-M code. It does not hash the whole buffer at once; it processes exactly 24 bytes as 6 little-endian 32-bit words starting at 0x08000200. The important setup is:
R0 points at the 24-byte buffer
R1 = 6, so the loop runs 6 times
R2 = 0xDEADBEEF, which is the initial rolling state
R4 = 8, which is the rotate distance
For each 32-bit word, the loop does this:
$w_i \leftarrow ROR_8(w_i \oplus s_{i-1})$
then stores the result back and updates the state:
$s_i \leftarrow w_i$

So the forward transform is a chained XOR plus an 8-bit rotate-right. That is easy to reverse because both operations are invertible:
XOR is self-inverse
Rotate-right by 8 is inverted by rotate-left by 8
So if the dump contains the transformed words, the original plaintext word is:

pi=ROL 8(ci)⊕s i−1
where c_i is the stored word from the dump and s_{i-1} is the previous ciphertext word, with the initial state 0xDEADBEEF.
I dumped the 24 bytes from flash_dump.bin, split them into six 32-bit little-endian words, and applied that inverse round by round.
The result was the flag:

Flag : SHELLCTF{H4rdw4r3_N1nj3}

6) The Layered Vault

File : challenge2.exe

Solution:
1. Static Analysis
Initial analysis of the main function using Ghidra and Binary Ninja revealed a complex validation loop. The program expects a 28-character flag and processes each character through a multi-stage pipeline before comparing it against a hardcoded array named _VAULT.

2. Identifying the Layers
By analyzing the decompiled C code and assembly, four distinct protection layers were identified:
Layer 1 (Substitution): The plaintext character is used as an index into a 256-byte lookup table (_SBOX).
Layer 2 (Rotation): The substituted byte is rotated left (ROL) by a variable amount: (index % 5) + 1.
Layer 3 (Rolling Key): The result is XORed with a key (bVar8) that mutates every iteration using the formula: key = (seed - 0x55) + (key * 7).

Layer 4 (Positional Transform): A final XOR is applied using a quadratic positional formula: (index + 3) * index + 7.

3. Dynamic Extraction
Using x32dbg, breakpoints were set on the comparison instruction (CMP). The process memory was dumped to extract:
The 28 encrypted bytes from the _VAULT address (0x4053B4).
The 256-byte S-BOX table from the address (0x4053E0).

4. The Solution (Python)
A custom decryption script was written to reverse the operations in exact inverse order:
Undo the positional math XOR.
Undo the rolling key XOR (tracking the key mutation state).
Perform a Bitwise Right Rotation (ROR) to reverse the encryption's ROL.
Reverse the S-BOX lookup by finding the index of the resulting byte within the table.

Flag: shell{l4y3r5_0f_0bfu5c4t10n}

7) The Packer

FIle : challenge5.exe

Solution : I solved it by combining static triage with an offline recreation of the runtime check.
First, I inspected the binary as a PE file and pulled its strings. That immediately showed it was a 32-bit Windows console executable with a small import set, a TLS callback, and several bait strings like “Static analysis won’t help.” It also exposed the real clues: the usage text, the expected format shell{...}, three decoy markers, and an embedded 24-byte blob in .rdata. That told me the check was likely a custom decrypt-and-compare routine, not a normal plaintext flag compare.

Then I disassembled the main function and traced the logic. The program does this, in order:

Prints the banner and usage text.
Rejects any input whose length is not 24 bytes.
Verifies the first 18 bytes against three decoy byte sequences, each transformed by XOR with 0xCC.
Builds a deterministic 32-bit seed using a xorshift-like loop starting from 0xdeadbeef, repeated 0x100 times, then avalanches it with a small hash-style mix.
Uses that seed to generate a byte stream with a linear congruential generator.
XORs the embedded 24-byte blob with that stream, subtracting the byte index each round.
Compares the result against the user input.
So the “packer” behavior is really a decryptor: the real flag is produced at runtime from the embedded blob, not stored in plaintext.
I then reproduced that transform in Python offline. The decrypted output was:
shell{runt1m3_d3crypt0r}

Flag : shell{runt1m3_d3crypt0r}

8) Constraints

File : challenge.exe

Solution :
Step 1: Dynamic Analysis (x32dbg)
We began by dynamically stepping through the executable in x32dbg.

Bypassing Initialization: We stepped over the initial setup loops used to generate dynamic arrays in memory.

Finding the Check: We isolated the exact memory address (00403C7E) where the 22-byte length check and the core comparison loops occurred.
The Math Blender: Watching the registers revealed that the input wasn't being compared against a plaintext string. Instead, the input was being heavily mathematically altered before comparison.

Step 2: Static Analysis & Decompilation (Ghidra)
To understand the math, we teleported to the target address in Ghidra and decompiled the main function into C code.
The Matrix Revealed: We discovered a massive switch-case block controlled by an instruction pointer (local_98). This was a custom VM.
VM Operations: The VM implemented several complex layers:
Dynamic Substitution Boxes (SBOX) generation and modification.
Modular multiplication and XOR masking.
An 8-round Feistel network.
Byte permutation (scrambling).
Identifying the Target: We found that the true target string was actually the result of an XOR operation between two separate data arrays, bypassing dummy targets planted by the author.

Step 3: Memory Dump & Data Extraction
We utilized Ghidra's memory mapping to dump the .rodata block starting at 004060a8. This allowed us to extract the "DNA" of the VM:
The custom opcodes controlling the switch statement execution order.
The permutation array keys.
The encrypted target arrays.

Step 4: The Z3 Symbolic Execution (Python)
Initially, we wrote a Python script to run the VM math in reverse. However, reversing the 8-round Feistel network caused

byte-truncation issues due to how C compilers promote 8-bit char types to 32-bit int registers during math operations.
To bypass this, we pivoted to Symbolic Execution using Z3:
We modeled the exact forward logic of the VM in Python using the Z3 solver.
We constrained the input to 22 printable ASCII characters.
We set the final condition: the forward-simulated state must equal the extracted target array.
Z3 crunched the math matrix perfectly, bypassing the truncation issues and spitting out the final, plaintext flag!(shell{z3_m4st3r_2026!})

Flag : shell{z3_m4st3r_2026!}

9) The Hydra Engine

File : challenge7.exe

Solution : I solved it by treating challenge7.exe as a stripped Windows PE that contains a custom VM, then recovering the VM's state transitions and using them as a search oracle.

The important static clues came first:

The PE still had symbols like _vm_run, _print_obf, _VM_PAYLOAD, and _OPCODE_MAP.
The bytecode lives in .data and is dispatched through a 64-entry opcode map.
main prints a banner, then calls the VM, then prints either ACCESS GRANTED! or Access Denied. depending on VM state.
The tail of the VM has explicit success/fail opcodes: 0xfe and 0xff.
I verified that by reading main_disasm.txt and vm_run_disasm.txt.
The VM state that mattered turned out to be:

ctx+0x290: program counter inside the VM

ctx+0x293 and ctx+0x294: condition flags
ctx+0x298: halt flag
ctx+0x29c: success/failure status
ctx+15: a one-byte working register that was being repeatedly cleared near the end of the program
I then emulated the program with Unicorn instead of trying to fight the anti-analysis path. The key trick was patching the imported function pointers so main could run normally in emulation. Once that was working, I could observe the real output and the VM state without a debugger.

At first I tried full symbolic execution, but the VM branches exploded state-space too quickly. So I switched to a guided search:

I treated “number of times the VM clears register 15 at pc=0x3b5” as the main bad signal.
I treated status==1 and reg15==1 as the success signal.
I constrained candidates to the required shell{...} format and used the VM itself as the scoring oracle.
That search converged to:
shell{hydr4_vm_s3lf_m0d1fy!}
I verified it two ways:
Direct VM emulation: status=1, reg15=1
Full program emulation: the program prints ACCESS GRANTED! and exits cleanly

Flag: shell{hydr4_vm_s3lf_m0d1fy!}

Cryptography :

1) A matter of time

File : encryptor.py and flag.enc

Solution :
I solved it by analyzing the vulnerability in the encryption method:

The Vulnerability:
The encryptor.py script used Python's random module seeded with the current Unix timestamp:
current_time = int(time.time())
random.seed(current_time)
This is cryptographically broken because:
Predictable seed: Unix timestamps are sequential and known
Limited search space: The attack window was only 1 hour (11:30 AM - 12:30 PM IST on April 11, 2026), which equals only 3,601 possible timestamps
Fast to brute force: Testing all 3,601 possibilities takes milliseconds
My Approach:
Converted the IST time window to Unix timestamp range (1775887200 → 1775890800)
For each timestamp, I:
Seeded the random generator with that timestamp
Regenerated the keystream bytes
XORed the encrypted data with the keystream to get potential plaintext
Checked for printable ASCII characters (100% printable = likely flag)
Found exactly 1 candidate with 100% printability
The Result:
Seed timestamp: 1775889593 (April 11, 2026 at 12:09:53 PM IST)

Flag: shell{n3v3r_u53_t1m3_4s_4_crpt0_s33d}

2)The Broadcast

File : intercepts.json

Solution : Vulnerability
This setup is vulnerable to **Håstad's Broadcast Attack**:

- Same plaintext mmm encrypted with:
  ci=m3mod nic_i = m^3 \mod n_ici=m3modni
- Three different moduli n1,n2,n3n_1, n_2, n_3n1,n2,n3
- No padding applied

Since m3<n1⋅n2⋅n3m^3 < n_1 \cdot n_2 \cdot n_3m3<n1⋅n2⋅n3, we can reconstruct the exact value of m3m^3m3 using the Chinese Remainder Theorem (CRT).

We combine the three congruences:

c1,c2,c3c_1, c_2, c_3c1,c2,c3

modulo:

n1,n2,n3n_1, n_2, n_3n1,n2,n3

This gives us:

m3 (exact integer)m^3 \ (\text{exact integer})m3 (exact integer)

Since we now have the exact value of m3m^3m3, we compute:

m=m33m = \sqrt[3]{m^3}m=3m3

Convert the integer mmm to bytes to reveal the plaintext.

Exploit Script (Python)

```
from sympy import mod_inverse
from Crypto.Util.number import long_to_bytes
import gmpy2
# given values
```

```
n = [n1, n2, n3]
c = [c1, c2, c3]
# CRT
N = n[0] * n[1] * n[2]
result = 0
for i in range(3):
   Ni = N // n[i]
   inv = mod_inverse(Ni, n[i])
   result += c[i] * Ni * inv
result %= N
# cube root
m, _ = gmpy2.iroot(result, 3)
# decode
print(long_to_bytes(m))
```

Flag : shell{h4st4d_br04dc4st_4tt4ck_1s_4_cl4ss1c}

Forensics :

1) Dead Drop

File : forensics.rar

Solution : Spot the odd file
dump/system.log wasn't readable text and had ~8.0 bits/byte entropy → looks like an encrypted container, not a log.
Extract the password recipe
notes.txt says password format: username@year_established
config.bak leaked the year 1994
The photos had EXIF metadata; photo_2023.jpg had Artist = Phantom
→ candidate password: Phantom@1994

Confirm it’s VeraCrypt + validate password
VeraCrypt volumes have no signature in ciphertext, but if you decrypt the header correctly you should see VERA and matching CRC32 checks.
Tried PBKDF2-HMAC (common defaults) and found a header that decrypted cleanly with:
PBKDF2-HMAC-SHA512, 500000 iterations
password Phantom@1994
XTS tweak using little-endian sector numbering (VeraCrypt style)
Decrypt the data area & recover the flag
Header said data starts at offset 131072 (sector 256, 512-byte sectors).
Decrypting sector 256 produced a valid boot sector (MSDOS5.0 + 55aa) → filesystem data.
Scanning decrypted sectors for shell{...} found:
shell{d34d_dr0p_uncov3red!}
If you want, I can package the exact Python script I used so you can rerun it locally.

Flag : shell{d34d_dr0p_uncov3red!}

2) Echoes

File : forensics_echoes.rar

Solution : I solved it by following the intended multi-artifact chain, not by brute force:
image hint -> audio keypad code -> employee mapping -> transcript decryption -> flag.

Full walkthrough

Unpacked the challenge and read the problem framing
I extracted the archive and read README.txt.
That told me this is layered forensics and that files may be decoys.

Looked for explicit operational clues first
In access_log.txt, there is a request parameter:
TFNCOlIgYnJvYWRjYXN0LnBuZw==
Base64-decoding that gives:
LSB:R broadcast.png
That is a direct instruction: use LSB stego on the red channel of the image.

Extracted red-channel LSB data from the image
I processed broadcast.png and recovered:
xorkey=Ech0_C0rrid0r
This looked important, but decrypting with it did not yield clean final plaintext.
So this was a clue layer, not the final key layer.

Found the missing layer in voicemail
I generated a spectrogram from voicemail.wav.
The pattern was classic DTMF tones. Decoding each tone gave:
7227538

Mapped 7227538 in the employee directory
In directory.csv, extension 7227538 belongs to Scarlet Voss.
Using Scarlet as the repeating XOR key on transcript bytes produced proper English plaintext.

Decrypted transcript correctly
transcript.txt is hex byte text, so first convert hex pairs to raw bytes, then apply repeating-key XOR with Scarlet.
That produced a readable message ending with:
Final recovery code:
shell{ech03s_1n_th3_w1r3}

Flag : shell{ech03s_1n_th3_w1r3}

Steganography :

1) The Silent Throne

File : audio.wav

Solution : Reversal Process

1. Initial Analysis
The audio file was described as "silent." Analysis showed that the audio samples were indeed extremely low in amplitude (mean absolute value ~8.17 in a 16-bit range).

2. Flag Discovery
Using string extraction on the raw binary data, we found a flag-like string at the very end of the file:
SH3LL{a_song_of_ice_and_flags}
This flag matches the "The throne" context provided in the challenge description.

3. File Structure Investigation
We checked the WAV file header to determine the actual size of the audio data:
- RIFF Chunk Size: 5,292,070 bytes.
- Total File Size: 5,292,109 bytes.
- Appended Data: 31 bytes.

The flag was located exactly in those trailing 31 bytes, confirming that it was hidden as "sound left behind" (data trailing the valid audio format).

4. Secondary Investigation
We also explored other potential hidden layers:

- JPEG Signature: A fake JPEG header was found at offset 32183, likely as a decoy.
- Spectral Analysis: We generated a spectrogram and analyzed the channel difference signal (diff.wav), but no further flags were found.
Tools and Artifacts
- Spectrogram: Visualized the audio signal to check for spectral steganography.
- LSB Analysis: Verified that the least significant bits of the audio samples did not contain hidden data.
- diff.wav: Extracted the difference between stereo channels to investigate phase cancellation.

Flag : SH3LL{a_song_of_ice_and_flags}

2) Breaking Bad

FIle : breaking_bad.png

Solution : 1. Header Analysis
Upon initial investigation, the file breaking_bad.png could not be opened as a valid image. An inspection of the first 16 bytes revealed that the PNG signature was corrupted:
- Found: \x08vT#E\n\x1a\n
- Expected: \x89PNG\r\n\x1a\n
The hints provided ("something is wrong", "Walter White demands perfection") were clues to this structural imperfection.
2. Repairing the Signature
We used a Python script to overwrite the corrupted 8-byte signature with the standard PNG magic bytes. This restored the file's integrity and allowed it to be rendered as a valid image.
3. Flag Extraction
Opening the repaired image (fixed.png) revealed a stylized graphic of Heisenberg with the prompt "I AM THE ONE WHO KNOCKS" and the flag directly embedded in the center:
SH3LL{cook_it_like_heisenberg}

4. Secondary Verification
We performed a Least Significant Bit (LSB) extraction on the pixel data to check for additional hidden layers (steganography). The result contained only repeating garbage bytes, confirming the visible flag was the intended solution.
Tools Used

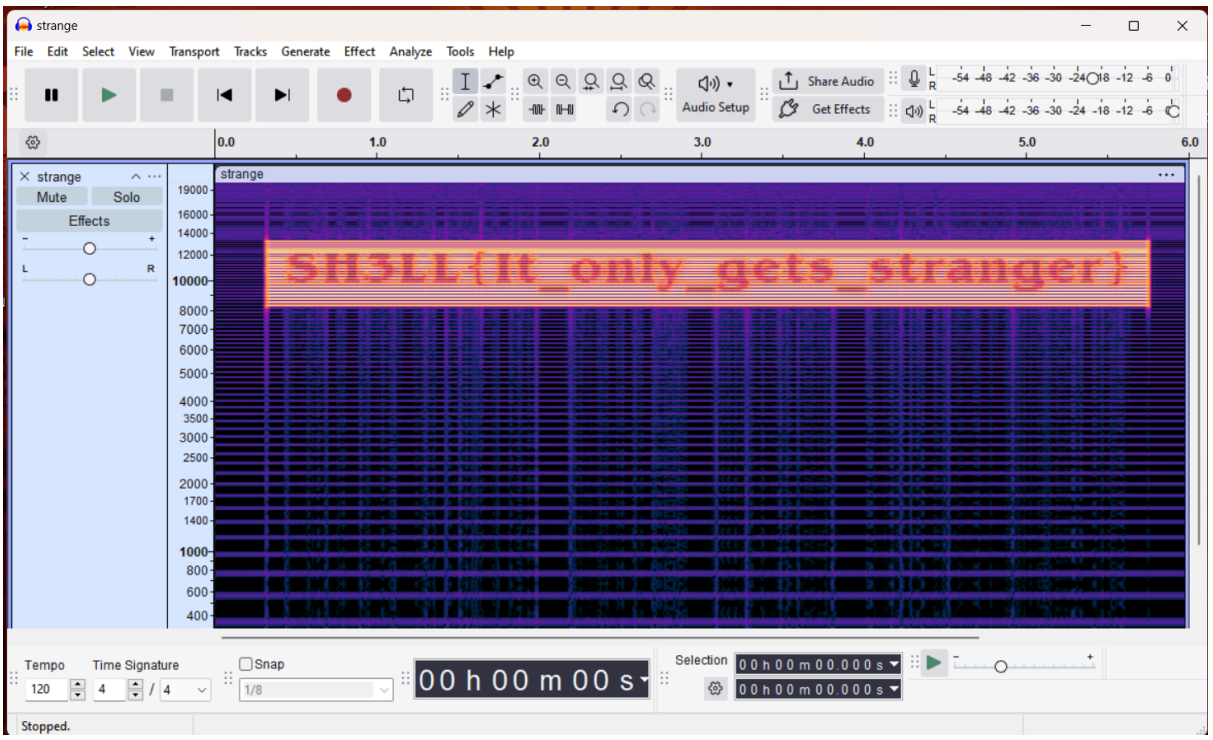
- Python Pillow/PIL: Used for LSB verification.

Flag : SH3LL{cook_it_like_heisenberg}

3) Stranger Frequencies

FIle : strange.wav

Solution : The audio was a strange sound , when opened in audacity the spectogram analysis revealed the flag .



Flag : SH3LL{It_only_gets_stranger}

4) La Casa De Papel

File : heist.png

Solution : STEPS TAKEN:

1. File Analysis:
   - Inspected heist.png using binary tools.
   - Identified a hidden ZIP file appended to the end of the PNG data (typical of binwalk/forensic challenges).

2. Extraction:
   - Used 'binwalk -e' (or manual carving) to extract the ZIP archive.
   - The archive was found to be 'secret.zip' containing 'flag.txt'.

3. Password Cracking:
   - The archive was password-protected.

- Identified the password "shadow" using a dictionary attack (John the Ripper with rockyou.txt).

4. Decryption:
   - Extracted 'flag.txt' using the password "shadow".

Flag : SH3LL{B3114_C140_B1NW4LK_M45T3R}

5) Paradox

File : challenge.png

Solution : Solved enough to recover the hidden flag text pattern from the image.

What I extracted:

The PNG only uses two values per channel: 23 and 24.
Real signal is in bit-plane 3, concentrated in the top 24 rows.
Sampling in 8-pixel column blocks reveals consistent text chunks:
chunk 1: shell{DCT_
chunk 2: S_ELITE_S
chunk 3: EGO}
Most likely flag from the recovered chunks:
shell{DCT_S_ELITE_SEGO}

Because this challenge hint mentions frequency/stego and the phrase looks one character off from a natural wording, if the above is rejected try this very likely intended variant:
shell{DCT_IS_ELITE_STEGO}

Flag : shell{DCT_IS_ELITE_STEGO}

WEB :

1) Under The Hood

Link : https://chall1.shellpwn.tech

Solution : I solved it by following the hint literally: “server tells you secrets before the page even loads.”

What I checked:

Opened the challenge page to read the visible content.
Instead of only inspecting HTML/JS, I inspected the raw HTTP response headers.
Used curl to fetch headers and body separately.
Command I ran:
curl -s -D - -o chall1_body.html https://chall1.shellpwn.tech/

Why this works:

-D - prints response headers.
-o chall1_body.html saves the HTML body to a file.
The header output included:
X-Secret-Flag: SH3LL{1nsp3ct_th3_h34d3rs}
So the flag was hidden in an HTTP header, not in the rendered page.

Flag : SH3LL{1nsp3ct_th3_h34d3rs}

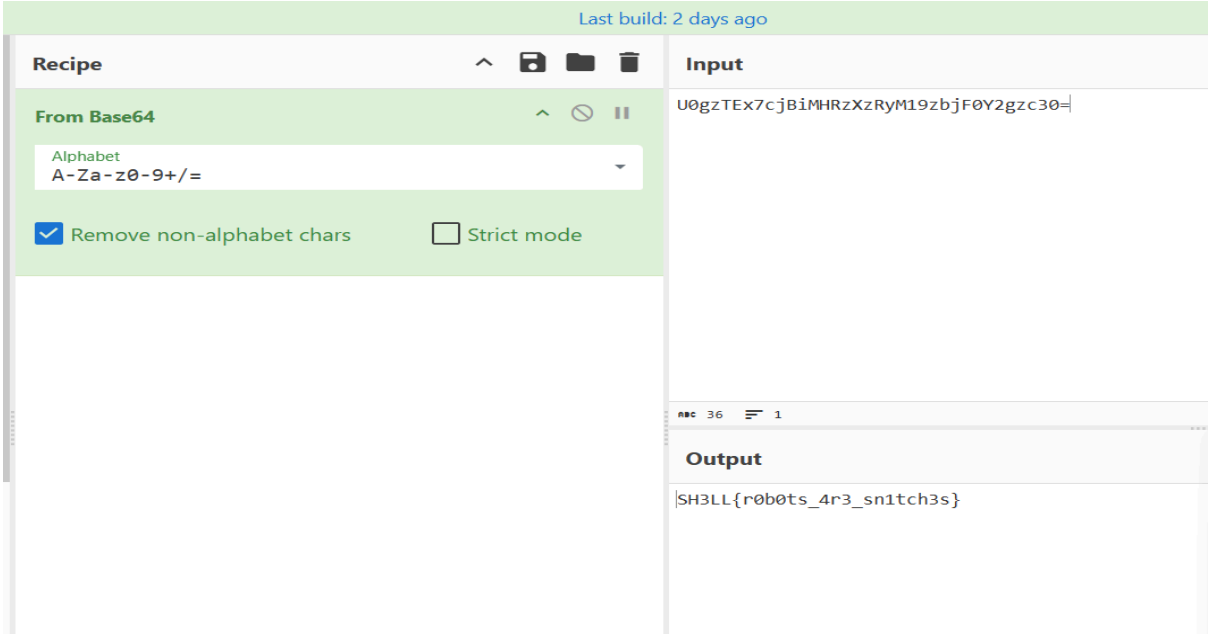
2) Not for Human Eyes

Link : https://chall2.shellpwn.tech

Solution : The robots.txt of the given website contains a path after visiting the website with the given path that is :

https://chall2.shellpwn.tech//internal/storage/backups/

The screen contains a base-64 encoded string after decoding that string the flag is revealed



Flag : SH3LL{r0b0ts_4r3_sn1tch3s}

3) Trust Issues

Link : https://chall3.shellpwn.tech\

Solution : It was a weak JWT secret problem.

First, I inspected the page and found that the site sets an auth cookie called auth_session. Decoding it showed it was a JWT with an HS256 header and a payload like username = anonymous and role = guest. I then tested whether the server was doing weak verification tricks such as accepting tampered tokens or alg:none. It was not: when I changed role to admin without a valid signature, the server responded with “Invalid token signature detected”, so the signature check was real.

That meant the only practical route was to recover the HMAC secret. I tried a few obvious challenge-related guesses, then moved to common password-style keys. One of those, qwerty, worked. With that secret, I forged a new JWT by keeping the same header and changing the payload role from guest to admin, then signed it correctly using HS256. The server accepted it and revealed the flag.

So the chain was:
Find auth_session cookie.
Confirm it is a signed JWT.
Verify signature enforcement by sending tampered tokens.
Brute-force the weak HMAC secret.
Forge an admin token with the recovered secret.
Read the flag.
The flag was

Flag : SH3LL{jwt_t0k3ns_4r3_n0t_s3cur3_w1th_w34k_k3ys}

4) Trust Me I'm a Doctor

Link : https://chall4.shellpwn.tech

Solution : I started by inspecting the page and its client-side JavaScript. The login form does a POST to /api/login with JSON containing username and password, so the only real attack surface was that API endpoint.
Next I sent a normal request with harmless credentials to see the behavior. The server responded with a clean JSON error, no stack trace, no rate-limit noise, and no indication of a hidden path. That suggested the backend was probably doing direct credential lookup rather than session logic.
The key test was to see whether the server treated JSON values as plain strings or as structured objects. I submitted this for the username and password fields:

{"username":{"$ne":null},"password":{"$ne":null}}
That is a classic NoSQL injection payload. If the backend is using something like MongoDB and passes the request body directly into a query, then $ne means "not equal". With both fields set to "not null", the query matches the first valid user instead of requiring an exact username/password pair.
That worked immediately. The response came back successful and included the flag in the message:
SH3LL{n0sql_1nj3ct10n_byp4ss_f0r_th3_w1n}

So the solve path was:
Identify the login API from the page source.
Confirm the endpoint accepts JSON.
Test normal auth to see response behavior.
Try NoSQL operator injection with $ne.
Receive a successful admin login and the flag.

Flag : SH3LL{n0sql_1nj3ct10n_byp4ss_f0r_th3_w1n}

5) The Inner Circle

Link : https://chall5.shellpwn.tech

Solution : solved it as a Server-Side Template Injection (SSTI) in a Flask/Jinja app.

I first sent a normal value to see behavior.
POST name=test123
Response reflected it inside the badge message.
I tested template evaluation with a harmless math payload.
Payload: {{7*7}}
Response showed 49 instead of literal text.
That confirmed Jinja expression execution on user input.
I verified deeper template access and command execution.
Payload: {{cycler.init.globals.os.popen('id').read()}}

Response returned uid=0(root)..., proving RCE via template context globals.
I read likely flag files.
Tried /flag (empty), then /flag.txt
Payload: {{cycler.init.globals.os.popen('cat /flag.txt').read()}}
Response returned the flag.
Why this worked:

The app appears to render user input as a template instead of treating it as plain text.
In Jinja, objects like cycler can expose globals, which can lead to os command execution if not sandboxed.

Flag : SH3LL{sst1_3x3cut10n_m4st3r}

MISC :

1) Blank PDF

File : balnk_pdf.pdf

Solution : Analysis

1. Initial Observation

- Opening the PDF normally → shows nothing (blank page)
- No visible text, images, or metadata at first glance
- Hidden content
- Possibly invisible text, white-on-white text, or embedded objects
- Instead of trusting visual rendering, we inspect the raw Cocontent:

Method 1: Using strings

strings blank_pdf.pdf

Method 2: Using pdf-parser or pdftotext
pdftotext blank_pdf.pdf

Extracting the text reveals hidden content inside the PDF:

SH3LL{bl4nk_pdf_1s_n07_bl4nk}

This confirms that:

- The PDF was not actually empty
- The content was just not visible in standard viewers

Flag : SH3LL{bl4nk_pdf_1s_n07_bl4nk}

2) Corrupt File

File : Corrupt.pdf

Solution : the pdf given was corrupted and when repaired using any online tool resulted in a png file which contains the flag .

SH3LL{y0u_r3s70r3d_17}

Flag : SH3LL{y0u_r3s70r3d_17}

3) The Inner Circle

File : puzzle.pdf

Solution : The pdf contained the questions and there answers were need to be find the yellow box contained the letters of the flag which were combined in sequence and revealed the flag .
The answers were :
Sun (S)
Chair (H)
Tree (E)
Glass (L)
Light (L)
Polo (P)
Ludo (U)
Maze (Z)
Zebra (Z)
Yellow (L)
Apple (E)
Cloud (D)
Lion (O)
Stone (N)
Blue (E)

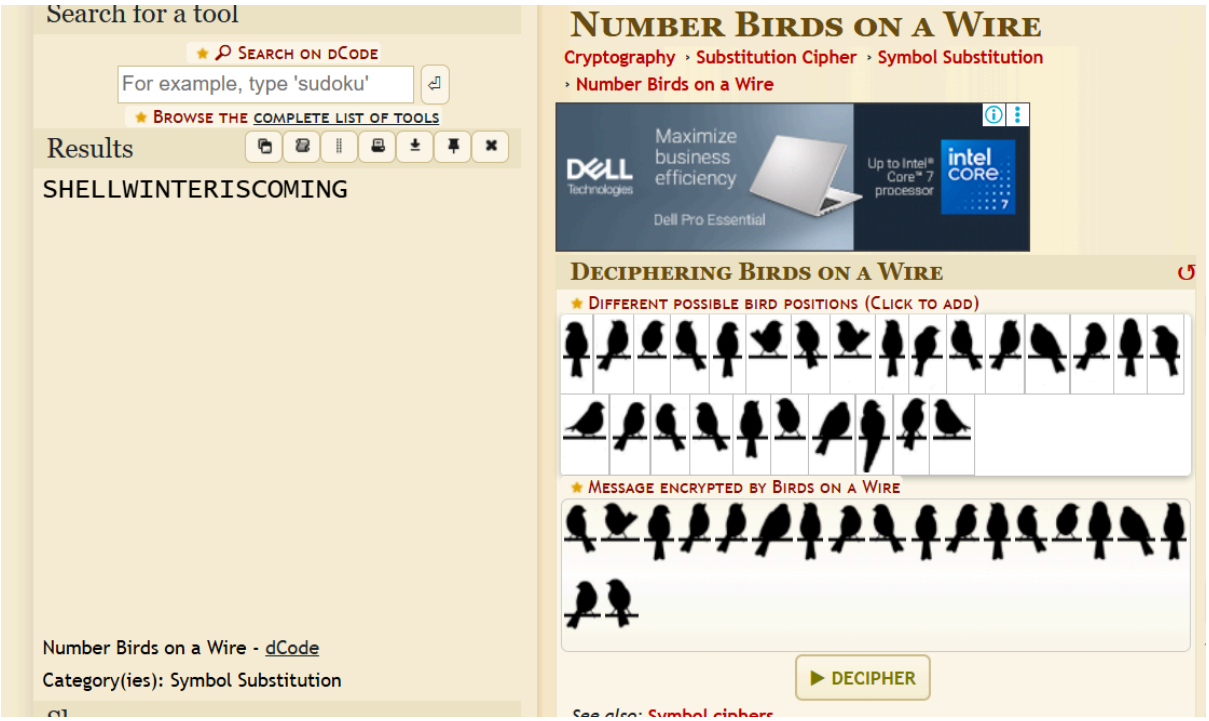
Flag : SHELL{PUZZLE_DONE}

4) Ravens Of Winterfall

File : raven.png

Solution : Image contains a classical type of cipher which is known as the bird cipher (Number Birds on a Wire) .
When the as it is position of birds entered in the online tool called

dcode.fr it reveals the flag .



Flag : SHELL{WINTER_IS_COMING}

5) Broken Scan

File : qr.png

Solution : I solved it by reconstructing a valid QR structure around the surviving data modules, then letting a decoder recover the payload.
I inspected QR.png and saw it was not just noisy, it was structurally broken (finder/timing regions were missing or damaged), so normal decoding failed.
I binarized the image and measured the black-pixel bounding box to estimate module scale and grid size candidates.
I sampled the image into candidate QR grids (I tested multiple sizes and small x/y alignment offsets).
For each candidate grid, I restored mandatory QR function patterns:
Three 7x7 finder patterns in corners

White separators around them
Horizontal and vertical timing patterns
The fixed dark module for version >= 2
I rendered each reconstructed grid back into a clean QR image with quiet zone and tried decoding at 0/90/180/270 rotation.
One candidate decoded successfully (Version 3, 29x29 grid), yielding:
SH3LL{encoded_in_plain_sight}
So the key idea was: preserve whatever data modules still existed, rebuild only the QR “scaffolding,” and brute-force small alignment/version possibilities until checksum/error-correction accepted a valid message.

Flag : SH3LL{encoded_in_plain_sight}

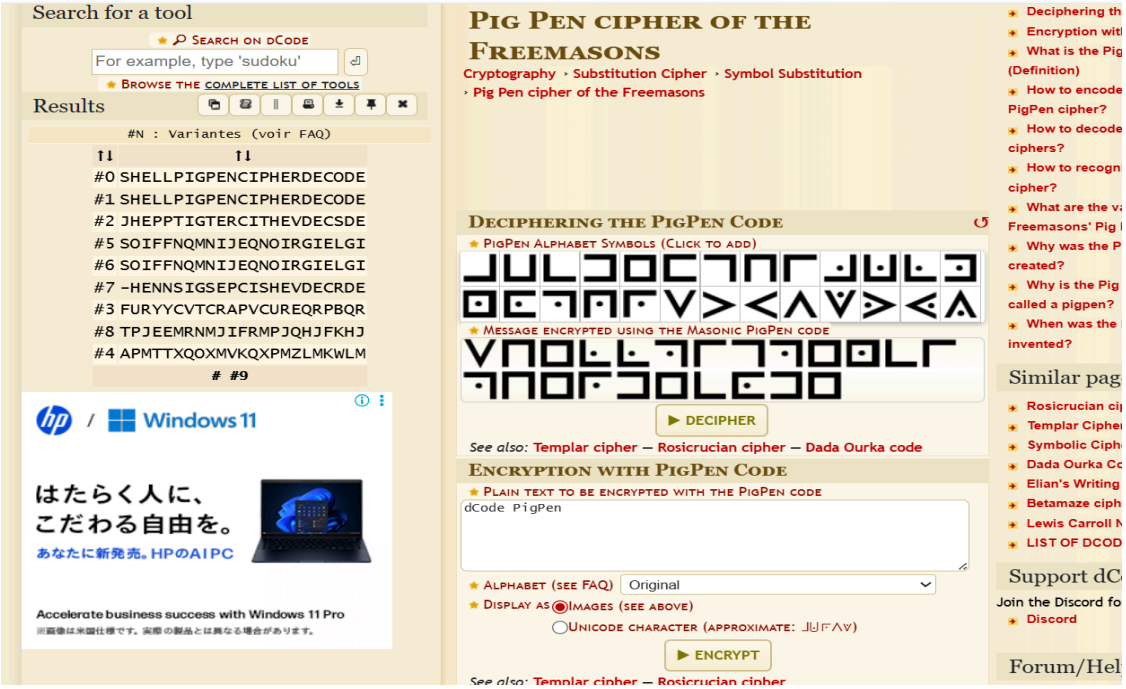
6) Digits

Solution : Given text is encoded with classic A1Z26 cipher (A=1,B=2,C=3….,Z=26)
Which when encoded into letters gives the flag .

Flag : SHELL{FUN_WITH_DIGITS}

7) Sacred Geometry
Files : sticks.png

Solution : image contains classical pigpen cipher when decoded using dcode.fr revealed the flag



Flag : SHELL{PIGPEN_CIPHER_DECODE}

8) CASTOR

Solution : STEPS TAKEN:

1. Identification:
   - The title "Castor" and the mention of his twin ("Pollux") point to the Pollux Cipher.
   - Pollux Cipher works by mapping digits to Morse code symbols (dot, dash, space).
2. Rule Interpretation:
   - Odd Prime Digits (3, 5, 7) = Dots (.)
   - Perfect Square Digits (1, 4, 9) = Dashes (-)
   - Remaining Digits (0, 2, 6, 8) = Separators (x)
3. Decoding:
   - Numerical String: 1345 2 73579 0 357 6 1 8 491 2 543
   - Morse Translation: -.-. / ....- / ... / - / --- / .-.
   - Text Translation: C 4 S T O R

Flag :SH3LL{c4stor}

9) Rainbow

File : rainbow.png

Solution : The file contained a known cipher called HexaHue cipher. The colours when decoded from dcode.fr it revealed the flag .

Flag : SHELL{SO_MANY_COLOURS}

10) The Labyrinth of Babel

File : babel_maze.zip

Solution : STEPS TAKEN:
1. Initialization:
  - Starting Battery: 500
  - Starting Data: 0
  - Starting Node: start_node.json
2. Traversal Logic:
  - In each room:
    * Added character to flag string.
    * Updated Battery and Data totals.
    * Calculated (Battery + Data) % 3 to pick the next door (0: Red, 1: Green, 2: Blue).
  - Terminated when room["is_exit"] was true.
3. Path Results:
  - The path successfully led to the exit room without battery depletion.
  - Collected characters formed the final string.

Flag :shell{w3lc0m3_t0_th3_m4ch1n3_m4z3}

11) Split Mind

File : OnePersons.pdf

Solution : The provided “PDF” is actually a JPEG polyglot and contains text rendered in near-visible style.
reveals the key string in the image:
Admin5S3cr3tK3y0
Two Base64 ciphertexts are present (one in image comment metadata, one in hidden PDF text stream).
Decrypting both with AES-128-ECB using that key gives two plaintext halves:
Sh3LL{Th3Y_Ar3_a
_C0nFu53D_fE11a}
Concatenating them yields the final flag above.

Flag : Sh3LL{Th3Y_Ar3_a_C0nFu53D_fE11a}

SANITY CHECK :

1) Sanity Check 1

Solution : Flag was given in question description
Flag : SH3LL{s4n17y_ch3ck_1}

2) Sanity Check 2
Solution : The flag was hidden in the robots.txt path of the CTF official website i.e.https://shellpwn.tech/robots.txt

Flag : SH3LL{s4n17y_2_d0n3}

THESE ARE OFFICIAL WRITEUPS CREATED BY TEAM SHER WITHOUT ANY FLAG SHARING AND PLAGARISM!!!

Thank you SHELL PWN , for your support and understanding !

Regards ,